# Rodrigo Fernandez

**Senior Rust Backend / System Engineer**
Katy, Texas | +1 (407) 801-1287 | www.linkedin.com/in/rodrigobermejo-fernández-9a5a0664 | rodrigobfdev@gmail.com

## Summary

Senior Rust Backend / Systems Engineer with fintech, healthcare, SaaS, cloud infrastructure, and data-platform experience, specializing in **Rust 1.56–1.88**, **Tokio 1.x**, **Axum 0.6–0.8**, **Actix Web 4.x**, PostgreSQL 12–16, Kafka 2.x–3.x, Docker, Kubernetes, and AWS to build low-latency APIs, resilient event-driven services, secure payment workflows, observability pipelines, and memory-safe migrations from Node.js, Java, and C++ systems while reducing production incidents, improving throughput, strengthening security controls, mentoring cross-functional teams, and turning complex legacy platforms into scalable, maintainable products for enterprise customers.

## Skills

- **Programming Languages: Rust 1.56–1.88**, TypeScript 4.x–5.x, JavaScript ES6+, Python 3.8–3.12, Go 1.18–1.22, C++11–C++17, SQL
- **Rust Backend: Tokio 1.x**, **Axum 0.6–0.8**, **Actix Web 4.x**, Hyper 0.14–1.x, Tonic 0.8–0.12, Tower, Serde, Reqwest, Clap, Rayon, Crossbeam, Anyhow, Thiserror
- **API & Microservices:** REST APIs, gRPC, WebSockets, OpenAPI, GraphQL Gateway Integration, Event-Driven Architecture, CQRS, Idempotency, Rate Limiting
- **Databases:** PostgreSQL 12–16, MySQL 8.x, Redis 6–7, DynamoDB, MongoDB 4–6, TimescaleDB, Elasticsearch 7–8, OpenSearch 1–2
- **Messaging & Streaming:** Kafka 2.x–3.x, RabbitMQ 3.x, AWS SQS/SNS, NATS, EventBridge, Dead-Letter Queues, Backpressure Handling
- **Cloud & DevOps:** AWS EC2, ECS, EKS, Lambda, S3, RDS, CloudWatch, IAM, Docker 20.x–25.x, Kubernetes 1.23–1.29, Terraform 1.x, GitHub Actions, GitLab CI
- **Testing & Quality:** Cargo Test, Criterion Benchmarking, Integration Testing, Contract Testing, Property-Based Testing, Load Testing, SonarQube, CodeQL
- **Security & Observability:** OAuth2, JWT, mTLS, Rustls, OpenTelemetry, Prometheus, Grafana, Loki, Jaeger, CloudWatch Logs, Secrets Manager, Vulnerability Scanning

## Experience

### Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Led the modernization of enterprise cloud infrastructure services by migrating latency-sensitive Node.js and Java APIs into **Rust 1.56–1.88**, **Tokio 1.x**, and **Axum 0.6–0.8**, improving request throughput by **38%** while reducing memory pressure during peak customer workloads.
- Designed secure multi-tenant backend services for SaaS and managed infrastructure customers using **Rust**, PostgreSQL 14–16, Redis 7, Kafka 3.x, and AWS services, allowing billing, provisioning, and customer-operations workflows to scale without frequent queue congestion.
- Resolved a recurring production issue where async workers silently stalled under heavy load by adding bounded channels, timeout guards, structured tracing, and retry policies, reducing stuck-job incidents by **42%** across critical background services.

- Migrated legacy REST endpoints from Express.js to **Axum 0.7–0.8** with OpenAPI contracts, typed request validation, and reusable middleware, improving API reliability while keeping frontend and partner integrations backward compatible.
- Implemented a high-performance usage-metering pipeline with **Tokio**, Kafka, PostgreSQL partitioning, and batch writes, reducing monthly invoice reconciliation time by **31%** and improving traceability for customer disputes.
- Built internal platform libraries for authentication, configuration loading, error mapping, and telemetry so multiple teams could use **Rust** consistently without rewriting boilerplate or creating incompatible service patterns.
- Fixed a memory growth issue caused by oversized payload buffering in a streaming ingestion service by replacing full-body reads with chunked processing, backpressure handling, and payload limits validated through load testing.
- Partnered with DevOps engineers to containerize **Rust** services with Docker, deploy them on Kubernetes, and tune CPU limits, readiness probes, and graceful shutdown behavior for safer rolling deployments.
- Introduced **OpenTelemetry**, Prometheus metrics, and Grafana dashboards across Rust microservices, giving support teams clearer visibility into p95 latency, queue lag, database saturation, and failed external calls.
- Improved security posture by integrating AWS Secrets Manager, least-privilege IAM roles, JWT validation, Rustls-based TLS configuration, and dependency scanning into CI/CD pipelines for regulated infrastructure workloads.
- Created migration playbooks for moving services from synchronous Node.js handlers to async Rust tasks, documenting ownership rules, error handling patterns, database transaction boundaries, and rollback strategies.
- Mentored backend engineers on **ownership**, lifetimes, async task design, and practical debugging with tracing spans, helping the team adopt Rust confidently without slowing feature delivery.

## NIX United —Software Engineer *(Oct 2016 – Mar 2020)*

- Developed fintech and logistics backend modules using **Rust 1.39–1.56**, **Actix Web 3.x–4.x**, PostgreSQL 11–13, Redis 5–6, and RabbitMQ, supporting transaction workflows that required strong consistency and predictable latency.
- Migrated selected Java Spring Boot services into **Rust** where CPU-bound validation and high-volume message processing created bottlenecks, improving batch processing speed by **34%** without changing external API contracts.
- Solved a duplicate-payment issue caused by retry storms by implementing idempotency keys, database uniqueness constraints, message deduplication, and explicit retry classification for transient versus permanent failures.
- Built REST and gRPC APIs with typed DTOs, Serde validation, middleware-based authorization, and structured error responses so frontend and mobile teams could integrate safely without ambiguous failure states.
- Implemented Kafka and RabbitMQ consumers with manual acknowledgment, dead-letter queues, exponential backoff, and poison-message isolation, reducing failed-event reprocessing noise by **29%** across payment workflows.
- Upgraded legacy Rust services from older async patterns to stable async-await with **Tokio 0.2–1.x**, simplifying worker logic and removing callback-heavy code that made debugging production incidents difficult.
- Optimized PostgreSQL queries by adding partial indexes, query-plan reviews, batch inserts, and connection-pool tuning, cutting average report-generation time by **26%** for operational dashboards.
- Resolved a hard-to-reproduce data race in a C++ integration component by moving the risky parsing layer into Rust with safer ownership boundaries and integration tests around malformed payloads.

- Implemented audit logging and role-based access control for sensitive financial actions, combining JWT claims, service-level permissions, and immutable database records for compliance review.
- Used **Criterion** benchmarks, flamegraphs, and load tests to compare serialization strategies, helping the team choose faster payload formats without sacrificing maintainability or debugging clarity.
- Collaborated with QA, product, and infrastructure teams to release Rust services through GitLab CI, Docker, and Kubernetes while keeping rollback procedures simple and production-safe.

### Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built e-commerce and digital-service backend features with PHP 7.x, Node.js 8–10, PostgreSQL 9–10, Redis, and early Rust 1.x utility services where performance-sensitive image and data-processing tasks needed safer execution.
- Introduced **Rust 1.20–1.36** command-line workers for CSV parsing, image metadata extraction, and scheduled data cleanup, reducing several nightly batch jobs by **24%** compared with previous scripting implementations.
- Fixed checkout failures caused by inconsistent inventory updates by adding transactional locking, retry-safe order states, and better reconciliation jobs between the application database and external payment providers.
- Migrated selected legacy PHP endpoints to Node.js services with typed validation, structured logging, and improved deployment packaging, helping the team separate unstable business logic from core storefront pages.
- Created internal Rust utilities with Clap, Serde, and Reqwest to automate catalog imports, supplier synchronization, and data normalization, making repeated operations faster and less error-prone for support teams.
- Solved a production issue where slow third-party API responses blocked worker queues by adding timeout handling, retry limits, Redis-backed job states, and operational alerts for delayed tasks.
- Improved database performance by rewriting heavy reporting queries, adding indexes, and archiving stale records, reducing dashboard load time by **21%** for business users during peak traffic periods.
- Worked flexibly across backend, infrastructure, and frontend integration tasks, connecting APIs to React interfaces, improving JSON contracts, and helping QA reproduce edge cases with realistic test data.
- Added Docker-based local environments for backend services so developers could reproduce payment, catalog, and cache-related bugs without relying on fragile shared development servers.
- Documented upgrade steps, environment variables, deployment checks, and rollback instructions, giving the team a more reliable path from older monolithic releases toward service-based delivery.

### Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Supported backend development for customer portals and internal admin tools using Java 7, PHP 5.x, MySQL 5.x, jQuery, Linux shell scripts, and early service-integration patterns common in business web applications.
- Built reporting endpoints, form workflows, and admin features while learning production debugging, database normalization, server logs, and practical release coordination with senior engineers.
- Fixed recurring session-expiration bugs by reviewing cookie settings, server-side session storage, and inconsistent timeout values across pages, improving user reliability for long-running admin tasks.
- Created SQL scripts and validation checks to clean duplicate customer records, repair inconsistent status values, and reduce manual support work caused by old data-entry problems.
- Improved legacy PHP modules by separating database logic from view templates, reducing repeated queries, and making future feature changes less risky for small business applications.
- Helped migrate older Java utility code toward cleaner Maven-based project structure, dependency version control, and repeatable build commands for easier team onboarding.

- Added basic monitoring scripts for disk space, failed cron jobs, and slow database backups, giving the team earlier warning before small operational issues became customer-facing outages.
- Worked closely with frontend developers to connect backend responses with jQuery-based interfaces, fixing validation mismatches and inconsistent error messages across customer-facing pages.
- Built a strong foundation in troubleshooting, version control, Linux environments, and production support that later made the transition into **Rust**, distributed systems, and cloud-native backend engineering natural.

### Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Supported internal engineering tools using **C#**, **.NET Framework 4.0–4.5**, **ASP.NET MVC 3–4**, JavaScript, jQuery, and SQL Server, gaining practical experience in enterprise software quality and structured development workflows.
- Assisted senior engineers with debugging UI defects, validation issues, and data-handling problems, documenting root causes clearly so fixes could be reviewed and released with lower regression risk.
- Built small internal web modules for tracking records, reviewing status changes, and improving team visibility, applying clean HTML, CSS, JavaScript, and backend integration patterns.
- Resolved early-stage bugs related to inconsistent form validation and SQL query results by reproducing issues locally, checking logs, and working with mentors to apply safer input-handling rules.

## Education

**Texas State University**
Bachelor's Degree in Computer Science (*Sep 2008 – May 2012)*